

1. Pandas Kutubxonasiga Kirish

- Pandas nima va uni o'rnatish.
- Pandas va NumPy o'rtasidagi farqlar.

2. Asosiy Ma'lumot Tuzilmalari

- **Series** va **DataFrame** yaratish va ulardan foydalanish.
- Indeks va ustunlar bilan ishlash.

3. Ma'lumotlar Bilan Ishlash

- CSV, Excel, JSON kabi formatlardan ma'lumotlarni o'qish va yozish.
- Filtrlar yordamida ma'lumotlarni tanlash.

4. Ma'lumotlarni O'zgartirish

- Yangi ustunlar qo'shish, ma'lumotlarni tozalash va dublikatlarni olib tashlash.
- Ma'lumot turlarini o'zgartirish.

5. Ma'lumotlarni Tahlil Qilish

- Deskriptiv statistikalar (o'rtacha, maksimal, minimal va boshqalar).
- `groupby()` va agregatsiyalar yordamida ma'lumotlarni guruhlash.

6. Ma'lumotlarni Vizualizatsiya Qilish

- Pandas va Matplotlib yordamida grafiklar yaratish.
- Seaborn bilan integratsiya.

7. Ilg'or Pandas Imkoniyatlari

- Vaqt bo'yicha ma'lumotlar bilan ishlash.
- Ma'lumotlarni birlashtirish va shakllantirish.

8. Pandasda Optimizatsiya va Samaradorlik

- Katta hajmdagi ma'lumotlar bilan ishlash.
- `categorical` tipidan foydalanish.

9. Mashqlar va Amaliyot

- Real hayotdan misollar yordamida mashqlar.
- Pandas yordamida ma'lumotlarni tahlil qilish bo'yicha kichik proyektlar.

10. Qayta Aloqa va Izlanish

- Pandas rasmiy dokumentatsiyasi.
- O'rganish davomida yuzaga kelgan savollarni onlayn forumlarda muhokama qilish.

1. Pandas Kutubxonasiga Kirish

Pandas nima?

Pandas — bu Python dasturlash tilida **ma'lumotlarni tahlil qilish va manipulyatsiya qilish** uchun mo'ljallangan kutubxona. Pandas yordamida siz ma'lumotlarni o'qish, tozalash, tahlil qilish, va ularni manipulyatsiya qilishni oson va samarali tarzda amalga oshirishingiz mumkin. Asosan, u **strukturalangan** va **vaqt bo'yicha** ma'lumotlar bilan ishlash uchun ishlatiladi.

Pandas ikkita asosiy ma'lumot tuzilmasini taklif etadi:

1. **Series** — bu bir o'lchovli ma'lumotlar tuzilmasi (masalan, ro'yxat yoki massiv).
2. **DataFrame** — bu ikki o'lchovli ma'lumotlar tuzilmasi, ya'ni jadval shaklida (ustunlar va qatorlar).

Pandasni o'rnatish

Pandas kutubxonasini Python muhitida o'rnatish juda oson. Agar sizda `pip` o'rnatilgan bo'lsa, quyidagi buyruqni bajarib o'rnatishingiz mumkin:

```
In [2]: pip install pandas
```

Collecting pandas

Downloading pandas-2.3.3-cp313-cp313-win_amd64.whl.metadata (19 kB)

Requirement already satisfied: numpy>=1.26.0 in c:\users\isakov\appdata\local\programs\python\python313\lib\site-packages (from pandas) (2.3.4)

Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\isakov\appdata\local\program s\python\python313\lib\site-packages (from pandas) (2.9.0.post0)

Collecting pytz>=2020.1 (from pandas)

Downloading pytz-2025.2-py2.py3-none-any.whl.metadata (22 kB)

Requirement already satisfied: tzdata>=2022.7 in c:\users\isakov\appdata\local\programs\python\python313\lib\site-packages (from pandas) (2025.2)

Requirement already satisfied: six>=1.5 in c:\users\isakov\appdata\local\programs\python\pytho n313\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

Downloading pandas-2.3.3-cp313-cp313-win_amd64.whl (11.0 MB)

```
----- 0.0/11.0 MB ? eta -:--:--
----- 0.0/11.0 MB ? eta -:--:--
- ----- 0.5/11.0 MB 1.9 MB/s eta 0:00:06
-- ----- 0.8/11.0 MB 2.9 MB/s eta 0:00:04
--- ----- 1.0/11.0 MB 1.6 MB/s eta 0:00:07
----- 1.6/11.0 MB 1.7 MB/s eta 0:00:06
----- 1.6/11.0 MB 1.7 MB/s eta 0:00:06
----- 2.1/11.0 MB 1.6 MB/s eta 0:00:06
----- 2.1/11.0 MB 1.6 MB/s eta 0:00:06
----- 2.4/11.0 MB 1.3 MB/s eta 0:00:07
----- 2.6/11.0 MB 1.4 MB/s eta 0:00:06
----- 2.9/11.0 MB 1.4 MB/s eta 0:00:06
----- 3.1/11.0 MB 1.3 MB/s eta 0:00:06
----- 3.4/11.0 MB 1.4 MB/s eta 0:00:06
----- 3.7/11.0 MB 1.4 MB/s eta 0:00:06
----- 3.9/11.0 MB 1.3 MB/s eta 0:00:06
----- 4.2/11.0 MB 1.3 MB/s eta 0:00:06
----- 4.2/11.0 MB 1.3 MB/s eta 0:00:06
----- 4.5/11.0 MB 1.3 MB/s eta 0:00:05
----- 4.7/11.0 MB 1.3 MB/s eta 0:00:05
----- 5.0/11.0 MB 1.3 MB/s eta 0:00:05
----- 5.2/11.0 MB 1.3 MB/s eta 0:00:05
----- 5.5/11.0 MB 1.3 MB/s eta 0:00:05
----- 5.8/11.0 MB 1.3 MB/s eta 0:00:05
----- 6.0/11.0 MB 1.3 MB/s eta 0:00:04
----- 6.0/11.0 MB 1.3 MB/s eta 0:00:04
----- 6.3/11.0 MB 1.2 MB/s eta 0:00:04
----- 6.3/11.0 MB 1.2 MB/s eta 0:00:04
----- 6.6/11.0 MB 1.2 MB/s eta 0:00:04
----- 6.6/11.0 MB 1.2 MB/s eta 0:00:04
----- 6.8/11.0 MB 1.2 MB/s eta 0:00:04
----- 7.1/11.0 MB 1.1 MB/s eta 0:00:04
----- 7.1/11.0 MB 1.1 MB/s eta 0:00:04
----- 7.3/11.0 MB 1.1 MB/s eta 0:00:04
----- 7.3/11.0 MB 1.1 MB/s eta 0:00:04
----- 7.6/11.0 MB 1.1 MB/s eta 0:00:04
----- 7.9/11.0 MB 1.1 MB/s eta 0:00:03
----- 7.9/11.0 MB 1.1 MB/s eta 0:00:03
----- 8.1/11.0 MB 1.1 MB/s eta 0:00:03
----- 8.4/11.0 MB 1.1 MB/s eta 0:00:03
----- 8.7/11.0 MB 1.1 MB/s eta 0:00:03
----- 8.9/11.0 MB 1.1 MB/s eta 0:00:02
----- 8.9/11.0 MB 1.1 MB/s eta 0:00:02
----- 9.2/11.0 MB 1.1 MB/s eta 0:00:02
----- 9.4/11.0 MB 1.1 MB/s eta 0:00:02
----- 9.7/11.0 MB 1.1 MB/s eta 0:00:02
----- 10.0/11.0 MB 1.1 MB/s eta 0:00:01
----- 10.2/11.0 MB 1.1 MB/s eta 0:00:01
```

[illegible]

Note: you may need to restart the kernel to use updated packages.

```
conda install pandas
```

O'rnatishdan so'ng, Pandasni Python faylida quyidagicha import qilishingiz mumkin:

Pandas va NumPy o'rtasidagi farqlar

1. Ma'lumot tuzilmalari:

- **NumPy** — bir o'lchovli (1D) yoki ikki o'lchovli (2D) massivlar bilan ishlaydi, bu esa ko'proq matematik hisob-kitoblar uchun qulay.
- **Pandas** — **Series** (bir o'lchovli) va **DataFrame** (ikki o'lchovli) kabi ma'lumot tuzilmalarini taklif qiladi. Pandasda har bir ustun turli ma'lumot turlarini (masalan, butun son, matn, va boshqa) saqlashi mumkin.

2. Indeksatsiya:

- **NumPy** massivlari indekslashda faqat sonli indekslarni qo'llaydi (0, 1, 2, ...).
- **Pandas** esa indekslashda faqat sonli emas, balki matnli indekslar va ustun nomlarini ham qo'llaydi.

3. Ma'lumotni boshqarish:

- **NumPy** faqat matematik operatsiyalar uchun mo'ljallangan va kengaytirilgan funksiyalarga ega emas.
- **Pandas** esa ko'plab qulay funksiyalarni taklif etadi, masalan, **ma'lumotlarni o'qish va yozish** (CSV, Excel, SQL), **ma'lumotlarni tozalash, guruhlash, statistik tahlil** va boshqalar.

4. Ma'lumotlarni tozalash va manipulyatsiya qilish:

- **Pandas** ma'lumotlar to'plamini oson tozalash va manipulyatsiya qilish imkonini beradi (masalan, bo'sh qiymatlarni to'ldirish yoki olib tashlash, dublikatlarni aniqlash va olib tashlash).
- **NumPy** esa asosan raqamli massivlar ustida ishlashga mo'ljallangan va uning ma'lumotlarni tozalash funksiyalari Pandasga qaraganda kamroq.

Misol:

- **NumPy** massivda faqat bitta turdagi ma'lumotlar bo'ladi, masalan:

```
In [3]: import numpy as np
```

```
In [5]: arr = np.array([1, 2, 3, 4])
```

- **Pandas** esa ustunlar turli ma'lumot turlarini o'z ichiga olishi mumkin:

```
In [5]: import pandas as pd
```

```
In [6]: data = {
    'Ism': ['Ali', 'Veli', 'Sami'],
    'Yosh': [23, 45, 34],
    'Shahar': ['Tashkent', 'Samarkand', 'Bukhara']
}
df = pd.DataFrame(data)
```

2. Asosiy Ma'lumot Tuzilmalari

Series yaratish va ulardan foydalanish

Series — bu bir o'lchovli ma'lumotlar tuzilmasi, u ko'plab ma'lumotlarni saqlash uchun ishlatiladi. Har bir **Series** obyekti indeksga ega bo'ladi, ya'ni har bir ma'lumot qiymatiga indeks (ko'rsatkich) beriladi.

Series yaratish

1. Ro'yxatdan Series yaratish:

```
In [7]: s = pd.Series([10, 20, 30, 40, 50])
        print(s)
```

```
0    10
1    20
2    30
3    40
4    50
dtype: int64
```

2. Kategoriyalangan ma'lumotlar bilan Series yaratish:

```
In [8]: s = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
        print(s)
```

```
a    10
b    20
c    30
d    40
dtype: int64
```

Bu yerda har bir ma'lumot qiymatiga indeks sifatida harflar (a, b, c, d) berilgan.

Series bilan ishlash

- Indeks orqali ma'lumotni olish:

```
In [9]: print(s['b']) # 'b' indeksidagi qiymat: 20
```

```
20
```

- Indeksni o'zgartirish:

```
In [10]: s.index = ['A', 'B', 'C', 'D']
         print(s)
```

```
A    10
B    20
C    30
D    40
dtype: int64
```

DataFrame yaratish va ulardan foydalanish

DataFrame — bu ikki o'lchovli, ya'ni jadval shaklida ma'lumotlar strukturasidir. Har bir DataFrame o'z ichiga **ustunlar** va **qatorlar** oladi.

DataFrame yaratish

1. Ro'yxatdan DataFrame yaratish:

```
In [7]: data = {
        'Ism': ['Ali', 'Vali', 'G\'ani'],
        'Yosh': [23, 45, 34],
        'Shahar': ['Tashkent', 'Samarkand', 'Bukhara']
        }
```

```
df = pd.DataFrame(data)
print(df)
```

	Ism	Yosh	Shahar
0	Ali	23	Tashkent
1	Vali	45	Samarkand
2	G'ani	34	Bukhara

2. Indexlar bilan DataFrame yaratish:

```
In [8]: df = pd.DataFrame(data, index=['row1', 'row2', 'row3'])
print(df)
```

	Ism	Yosh	Shahar
row1	Ali	23	Tashkent
row2	Vali	45	Samarkand
row3	G'ani	34	Bukhara

Bu yerda DataFrame'ning har bir qatori uchun maxsus indekslar (`row1` , `row2` , `row3`) berilgan.

DataFrame bilan ishlash

- **Ustunga murojaat qilish:**

```
In [9]: print(df['Ism']) # 'Ism' ustunidagi barcha ma'lumotlar
```

```
row1    Ali
row2    Vali
row3    G'ani
Name: Ism, dtype: object
```

- **Bir nechta ustunlarni tanlash:**

```
In [10]: print(df[['Ism', 'Shahar']])
```

	Ism	Shahar
row1	Ali	Tashkent
row2	Vali	Samarkand
row3	G'ani	Bukhara

- **Qatorlarni tanlash:**

- **`loc[]` yordamida** (indeks nomlari bo'yicha):

```
In [11]: print(df.loc['row1']) # 'row1' qatoridagi barcha ma'lumotlar
```

```
Ism      Ali
Yosh     23
Shahar   Tashkent
Name: row1, dtype: object
```

- **`iloc[]` yordamida** (raqamli indekslar bo'yicha):

```
In [12]: print(df.iloc[0]) # Birinchi qator
```

```
Ism            Ali
Yosh           23
Shahar         Tashkent
Name: row1, dtype: object
```

Indeks va Ustunlar Bilan Ishlash

- Indeksni olish:

```
In [17]: print(df.index) # Qatorlar indekslarini olish
```

```
Index(['row1', 'row2', 'row3'], dtype='object')
```

- Ustun nomlarini olish:

```
In [18]: print(df.columns) # Ustun nomlarini olish
```

```
Index(['Ism', 'Yosh', 'Shahar'], dtype='object')
```

- Indeksni o'zgartirish:

```
In [13]: df.index = ['A', 'B', 'C']
print(df)
```

	Ism	Yosh	Shahar
A	Ali	23	Tashkent
B	Vali	45	Samarkand
C	G'ani	34	Bukhara

3. Ma'lumotlar Bilan Ishlash

CSV, Excel, JSON kabi formatlardan ma'lumotlarni o'qish va yozish

Pandas kutubxonasi yordamida turli formatdagi ma'lumotlarni o'qish va yozish juda oson. Quyidagi misollarda ma'lumotlar bilan ishlashni ko'rib chiqamiz:

CSV formatidan ma'lumotlarni o'qish

CSV (Comma Separated Values) — bu eng keng tarqalgan ma'lumotlar formati bo'lib, Pandas yordamida uni o'qish uchun `read_csv()` metodidan foydalanamiz:

```
In [18]: data = {
    'Ism': ['Ali', 'Vali', 'G\'ani'],
    'Yosh': [23, 45, 34],
    'Shahar': ['Tashkent', 'Samarkand', 'Bukhara']
}

df = pd.DataFrame(data)
df.to_csv('data.csv', index=False)
```

```
In [19]: # CSV faylini o'qish
df = pd.read_csv('data.csv')

# Natijani ko'rish
print(df)
```


	Ism	Yosh	Shahar
0	Ali	23	Tashkent
1	Vali	45	Samarkand
2	G'ani	34	Bukhara

`read_csv()` metodi faylni o'qib, uning tarkibini **DataFramega** aylantiradi. Agar CSV faylida maxsus ajratgichlar bo'lsa (masalan, `,`), u holda `sep` parametridan foydalanish mumkin:

```
In [25]: df = pd.read_csv('data.csv', sep=',')
print(df)
```

	Ism	Yosh	Shahar
0	Ali	23	Tashkent
1	Veli	45	Samarkand
2	Sami	34	Bukhara

Excel formatidan ma'lumotlarni o'qish

Pandas yordamida Excel fayllarini o'qish uchun `read_excel()` metodidan foydalanamiz. Excel faylini o'qishdan oldin, `openpyxl` yoki `xlrd` kabi kutubxonalarni o'rnatish kerak bo'ladi:

```
In [27]: pip install openpyxl
```

```
Collecting openpyxl
  Downloading openpyxl-3.1.5-py2.py3-none-any.whl.metadata (2.5 kB)
Collecting et-xmlfile (from openpyxl)
  Downloading et_xmlfile-2.0.0-py3-none-any.whl.metadata (2.7 kB)
Downloading openpyxl-3.1.5-py2.py3-none-any.whl (250 kB)
Downloading et_xmlfile-2.0.0-py3-none-any.whl (18 kB)
Installing collected packages: et-xmlfile, openpyxl
```

```
----- 1/2 [openpyxl]
----- 1/2 [openpyxl]
----- 2/2 [openpyxl]
```

```
Successfully installed et-xmlfile-2.0.0 openpyxl-3.1.5
Note: you may need to restart the kernel to use updated packages.
```

Excel faylini o'qish:

```
In [20]: # Ma'lumotlar yaratish
data = {
    'Ism': ['Ali', 'Vali', 'G\'ani'],
    'Yosh': [23, 45, 34],
    'Shahar': ['Tashkent', 'Samarkand', 'Bukhara']
}

df = pd.DataFrame(data)

# DataFrame'ni Excel faylga yozish
df.to_excel('data.xlsx', index=False)
```

```
In [31]: df = pd.read_excel('data.xlsx', sheet_name='Sheet1')
print(df)
```

	Ism	Yosh	Shahar
0	Ali	23	Tashkent
1	Veli	45	Samarkand
2	Sami	34	Bukhara

`sheet_name` parametri orqali ma'lum bir varoqni o'qish mumkin.

JSON formatidan ma'lumotlarni o'qish

JSON (JavaScript Object Notation) — ma'lumotlarni saqlash uchun ishlatiladigan eng keng tarqalgan formatlardan biri. Pandasda JSON formatidan foydalanish uchun `read_json()` metodidan foydalanamiz:

```
In [24]: # Yangi ma'lumotlarni yaratish
data = {
    'Ism': ['Ali', 'Vali', 'G'ani'],
    'Yosh': [23, 45, 34],
    'Shahar': ['Tashkent', 'Samarkand', 'Bukhara']
}

# DataFrame yaratish
df = pd.DataFrame(data)

# DataFrame'ni JSON faylga yozish
df.to_json('data.json', orient='records', lines=True)
```

```
In [23]: # JSON faylni o'qish
df_read = pd.read_json('data.json', lines=True)
print(df_read)
```

	Ism	Yosh	Shahar
0	Ali	23	Tashkent
1	Vali	45	Samarkand
2	G'ani	34	Bukhara

Ma'lumotlarni turli formatlarga yozish

Ma'lumotlarni **CSV**, **Excel**, yoki **JSON** formatlariga yozish ham oson. Quyidagi misollarni ko'rib chiqamiz:

1. CSV formatiga yozish:

0 Namunaviy DataFrame

```
In [45]: data = {
    "name": ["Ali", "Vali", "Hasan", "Husan"],
    "age": [25, 30, 28, np.nan],
    "salary": [3500.5, 4200.75, 3900.0, 4100.125],
    "city": ["Toshkent", "Samarqand", "Buxoro", "Toshkent"],
    "join_date": pd.to_datetime(["2024-01-15", "2023-06-01", "2022-12-10", "2024-02-20"])
}

df = pd.DataFrame(data)
df
```

```
Out[45]:
```

	name	age	salary	city	join_date
0	Ali	25.0	3500.500	Toshkent	2024-01-15
1	Vali	30.0	4200.750	Samarqand	2023-06-01
2	Hasan	28.0	3900.000	Buxoro	2022-12-10
3	Husan	NaN	4100.125	Toshkent	2024-02-20

1 path_or_buf – saqlanadigan joy yoki matn sifatida qaytarish

Tavsif

- Fayl nomi yoki to'liq yo'l ('output.csv' , 'D:/data/output.csv' va hokazo).
- Agar None bo'lsa, CSV matn ko'rinishida **qaytaradi** (diskka yozmaydi).

Misollar

```
# 1. Oddiy holat: shu joriy papkaga yozish
df.to_csv('output_basic.csv') # index=True bo'yicha default yoziladi
# 2. To'liq yo'l bilan yozish (Windows misolida)
df.to_csv(r'D:\csv_files\output_fullpath.csv')
```

```
In [31]: # 3. Faylga emas, matn sifatida olish
csv_matn = df.to_csv(None, index=False)
print(csv_matn)
```

```
Ism,Yosh,Shahar
Ali,23,Tashkent
Vali,45,Samarkand
G'ani,34,Bukhara
```

```
In [33]: # 4. Faylga emas, matn sifatida olish (index = True bo'lsa)
csv_matn = df.to_csv(None, index=True)
print(csv_matn)
```

```
,Ism,Yosh,Shahar
0,Ali,23,Tashkent
1,Vali,45,Samarkand
2,G'ani,34,Bukhara
```

2 sep – ustun ajratgich (delimiter)

Tavsif

- Default – ',' (vergul).
- TSV (\t), ; yoki boshqa belgi bilan ajratilgan formatlar uchun ishlatiladi.

Misollar

```
In [34]: # 1. Standart CSV (vergul bilan)
csv_matn = df.to_csv(None, index=True)
print(csv_matn)
```

```
,Ism,Yosh,Shahar
0,Ali,23,Tashkent
1,Vali,45,Samarkand
2,G'ani,34,Bukhara
```

```
In [36]: # 2. Nuqtali vergul bilan
csv_matn = df.to_csv(None, sep=';', index=False)
print(csv_matn)
```

```
Ism;Yosh;Shahar
Ali;23;Tashkent
Vali;45;Samarkand
G'ani;34;Bukhara
```

```
In [38]: # 3. TSV fayl (tab bilan ajratilgan)
csv_matn = df.to_csv(None, sep='\t', index=False)
print(csv_matn)
```

```
Ism      Yosh      Shahar
Ali       23       Tashkent
Vali      45       Samarkand
G'ani     34       Bukhara
```

3 index – qator indekslarini yozish/yozmaslik

Tavsif

- `True` – indeks CSVga yoziladi (default).
- `False` – indeks yozilmaydi, faqat ustunlar.

Misollar

```
In [40]: # 1. IndeksLarni yozmasdan saqlash
csv_matn = df.to_csv(None, index=False)
print(csv_matn)
```

```
Ism,Yosh,Shahar
Ali,23,Tashkent
Vali,45,Samarkand
G'ani,34,Bukhara
```

```
In [42]: # 2. IndeksLar bilan saqlash (defaultga teng)
csv_matn = df.to_csv(None, index=True)
print(csv_matn)
```

```
,Ism,Yosh,Shahar
0,Ali,23,Tashkent
1,Vali,45,Samarkand
2,G'ani,34,Bukhara
```

4 header – ustun nomlarini yozish/yozmaslik

Tavsif

- `True` – ustun nomlarini birinchi qatorda yozadi.
- `False` – ustun nomlari yozilmaydi.
- `header=['A', 'B', ...]` – ustun nomlarini o'zgartirib yozish.

Misollar

In [43]: *# 1. Ustun nomlarisiz CSV*

```
csv_matn = df.to_csv(None, index=False, header=False)
print(csv_matn)
```

Ali,23,Tashkent
Vali,45,Samarkand
G'ani,34,Bukhara

In [46]: *# 2. Ustun nomlarini boshqacha qilib berish*

```
csv_matn = df.to_csv(
    None,
    index=False,
    header=['Ism', 'Yosh', 'Maosh', 'Shahar', 'Ishga_kirgan_sana']
)
print(csv_matn)
```

Ism,Yosh,Maosh,Shahar,Ishga_kirgan_sana
Ali,25.0,3500.5,Toshkent,2024-01-15
Vali,30.0,4200.75,Samarqand,2023-06-01
Hasan,28.0,3900.0,Buxoro,2022-12-10
Husan,,4100.125,Toshkent,2024-02-20

5 columns – faqat tanlangan ustunlarni yozish

Tavsif

- Ro'yxat ko'rinishida beriladi.
- Faqat shu ustunlar CSVga tushadi.

Misollar

In [47]: *# 1. Faqat ism va yosh*

```
csv_matn = df.to_csv(None, index=False, columns=['name', 'age'])
print(csv_matn)
```

```
name,age
Ali,25.0
Vali,30.0
Hasan,28.0
Husan,
```

In [48]: *# 2. Faqat ism, maosh va shahar*

```
csv_matn = df.to_csv(None, index=False, columns=['name', 'salary', 'city'])
print(csv_matn)
```

```
name,salary,city
Ali,3500.5,Toshkent
Vali,4200.75,Samarqand
Hasan,3900.0,Buxoro
Husan,4100.125,Toshkent
```

6 na_rep – NaN qiymatlarni almashtirish

Tavsif

- `NaN` / bo'sh qiymatlar CSVda qanday yozilishi kerakligini belgilaydi.

Misollar

```
In [51]: data = {
    "name": ["Ali", None, "Hasan", "Husan"],
    "age": [25, 30, 28, np.nan],
    "salary": [3500.5, 4200.75, 3900.0, 4100.125],
    "city": ["Toshkent", "Samarqand", None, "Toshkent"],
    "join_date": pd.to_datetime(["2024-01-15", "2023-06-01", "2022-12-10", None])
}

df = pd.DataFrame(data)
df
```

Out[51]:

	name	age	salary	city	join_date
0	Ali	25.0	3500.500	Toshkent	2024-01-15
1	None	30.0	4200.750	Samarqand	2023-06-01
2	Hasan	28.0	3900.000	None	2022-12-10
3	Husan	NaN	4100.125	Toshkent	NaT

```
In [56]: # 1. Bo'sh qiymatlar o'rniga 'yo'q' yozish
csv_matn = df.to_csv(None, sep='\t', index=False, na_rep='yo'q')
print(csv_matn)
```

name	age	salary	city	join_date
Ali	25.0	3500.5	Toshkent	2024-01-15
yoʻq	30.0	4200.75	Samarqand	2023-06-01
Hasan	28.0	3900.0	yoʻq	2022-12-10
Husan	yoʻq	4100.125	Toshkent	yoʻq

In [57]: *# 2. Bo'sh qiymatlar o'rniga 'N/A' yozish*

```
csv_matn = df.to_csv(None, sep='\\t', index=False, na_rep='N/A')
print(csv_matn)
```

name	age	salary	city	join_date
Ali	25.0	3500.5	Toshkent	2024-01-15
N/A	30.0	4200.75	Samarqand	2023-06-01
Hasan	28.0	3900.0	N/A	2022-12-10
Husan	N/A	4100.125	Toshkent	N/A

7 float_format – oʻnli sonlarni formatlash

Tavsif

- C / printf uslubidagi format: masalan '%.2f', '%.3f' va hokazo.

Misollar

In [59]: *# 1. Maoshni verguldan keyin 2 ta son bilan yozish*

```
csv_matn = df.to_csv(None, index=False, float_format='%.2f')
print(csv_matn)
```

```
name,age,salary,city,join_date
Ali,25.00,3500.50,Toshkent,2024-01-15
,30.00,4200.75,Samarqand,2023-06-01
Hasan,28.00,3900.00,,2022-12-10
Husan,,4100.12,Toshkent,
```

In [60]: *# 2. Maoshni 3 ta kasr bilan yozish*

```
csv_matn = df.to_csv(None, index=False, float_format='%.3f')
print(csv_matn)
```

```
name,age,salary,city,join_date
Ali,25.000,3500.500,Toshkent,2024-01-15
,30.000,4200.750,Samarqand,2023-06-01
Hasan,28.000,3900.000,,2022-12-10
Husan,,4100.125,Toshkent,
```

8 encoding – fayl kodlash formati

Tavsif

- Keng tarqalgan variantlar: `utf-8` , `utf-8-sig` , `latin1` , `cp1251` , `cp1252` .
- Windows Excel ko'pincha `utf-8-sig` bilan yaxshi ochadi.

Misollar

In [61]: *# 1. UTF-8 bilan saqlash*

```
csv_matn = df.to_csv(None, index=False, encoding='utf-8')
print(csv_matn)
```

```
name,age,salary,city,join_date
Ali,25.0,3500.5,Toshkent,2024-01-15
,30.0,4200.75,Samarqand,2023-06-01
Hasan,28.0,3900.0,,2022-12-10
Husan,,4100.125,Toshkent,
```

In [62]: *# 2. Excel uchun qulay UTF-8-SIG bilan saqlash*

```
csv_matn = df.to_csv(None, index=False, encoding='utf-8-sig')
print(csv_matn)
```

```
name,age,salary,city,join_date
Ali,25.0,3500.5,Toshkent,2024-01-15
,30.0,4200.75,Samarqand,2023-06-01
Hasan,28.0,3900.0,,2022-12-10
Husan,,4100.125,Toshkent,
```

In [63]: *# 3. Ruscha/Lotin aralash matnlar uchun ba'zan Latin1 ishlatiladi*

```
csv_matn = df.to_csv(None, index=False, encoding='latin1')
print(csv_matn)
```

```
name,age,salary,city,join_date
Ali,25.0,3500.5,Toshkent,2024-01-15
,30.0,4200.75,Samarqand,2023-06-01
Hasan,28.0,3900.0,,2022-12-10
Husan,,4100.125,Toshkent,
```

9 lineterminator – qator tugash belgilari

Tavsif

- Har bir satr oxirida ishlatiladigan belgi: `\n` , `\r\n` va hokazo.
- Platformaga bog'liq muammolar bo'lsa (Linux/Windows), foydali bo'ladi.

Misollar

In [67]: *# 1. Faqat yangi qator belgisi bilan (Unix uslubida)*

```
csv_matn = df.to_csv(None, index=False, lineterminator='\n')
print(csv_matn)
```



```
name,age,salary,city,join_date
Ali,25.0,3500.5,Toshkent,2024-01-15
,30.0,4200.75,Samarqand,2023-06-01
Hasan,28.0,3900.0,,2022-12-10
Husan,,4100.125,Toshkent,
```

```
In [69]: # 2. Windows uslubi (ko'p hollarda default o'zi shunaqa bo'ladi)
csv_matn = df.to_csv(None, index=False, lineterminator='\r\n')
print(csv_matn)
```

```
name,age,salary,city,join_date
Ali,25.0,3500.5,Toshkent,2024-01-15
,30.0,4200.75,Samarqand,2023-06-01
Hasan,28.0,3900.0,,2022-12-10
Husan,,4100.125,Toshkent,
```

10 quotechar – matnlarni oʻrash uchun ishlatiladigan belgi

Tavsif

- Masalan, "Ali, Vali" degan matn boʻlsa, CSV ichida "Ali, Vali" tarzida saqlash kerak boʻladi.
- Default: " (qoʻshtirnoq).

Misollar

```
In [72]: data = {
    "name": ["Ali, Vali", None, "Hasan", "Husan"],
    "age": [25, 30, 28, np.nan],
    "salary": [3500.5, 4200.75, 3900.0, 4100.125],
    "city": ["Toshkent", "Samarqand", "Andijon, Toshkent", "Toshkent"],
    "join_date": pd.to_datetime(["2024-01-15", "2023-06-01", "2022-12-10", None])
}

df = pd.DataFrame(data)
df
```

```
Out[72]:
```

	name	age	salary	city	join_date
0	Ali, Vali	25.0	3500.500	Toshkent	2024-01-15
1	None	30.0	4200.750	Samarqand	2023-06-01
2	Hasan	28.0	3900.000	Andijon, Toshkent	2022-12-10
3	Husan	NaN	4100.125	Toshkent	NaT

```
In [73]: # 1. Standart qo'shtirnoq bilan
csv_matn = df.to_csv(None, index=False, quotechar='"')
print(csv_matn)
```

```
name,age,salary,city,join_date
"Ali, Vali",25.0,3500.5,Toshkent,2024-01-15
,30.0,4200.75,Samarqand,2023-06-01
Hasan,28.0,3900.0,"Andijon, Toshkent",2022-12-10
Husan,,4100.125,Toshkent,
```

```
In [74]: # 2. Bitta tirnoq bilan o'rash
csv_matn = df.to_csv(None, index=False, quotechar="'")
print(csv_matn)
```

```
name,age,salary,city,join_date
'Ali, Vali',25.0,3500.5,Toshkent,2024-01-15
,30.0,4200.75,Samarqand,2023-06-01
Hasan,28.0,3900.0,'Andijon, Toshkent',2022-12-10
Husan,,4100.125,Toshkent,
```

1 1 quoting – qaysi hollarda qo'shtirnoq ishlatish

Tavsif

csv modulidagi konstanlar bilan ishlaydi:

```
In [76]: import csv
```

- `csv.QUOTE_MINIMAL` – kerak bo'lgandagina (default).
- `csv.QUOTE_ALL` – hamma maydonlarni qo'shtirnoqqa oladi.
- `csv.QUOTE_NONNUMERIC` – faqat matn/NaN ustunlar qo'shtirnoqda.
- `csv.QUOTE_NONE` – umuman qo'shtirnoq ishlatmaydi (diqqat: `escapechar` talab qilishi mumkin).

Misollar

```
In [77]: # 1. Hamma maydonlarni qo'shtirnoqqa olish
csv_matn = df.to_csv(None, index=False, quoting=csv.QUOTE_ALL)
print(csv_matn)
```

```
"name","age","salary","city","join_date"
"Ali, Vali","25.0","3500.5","Toshkent","2024-01-15"
","30.0","4200.75","Samarqand","2023-06-01"
"Hasan","28.0","3900.0","Andijon, Toshkent","2022-12-10"
"Husan","","4100.125","Toshkent",""
```

```
In [78]: # 2. Faqat matn tipidagi ustunlarni qo'shtirnoqqa olish
csv_matn = df.to_csv(None, index=False, quoting=csv.QUOTE_NONNUMERIC)
print(csv_matn)
```

```
"name","age","salary","city","join_date"
"Ali, Vali",25.0,3500.5,"Toshkent","2024-01-15"
","30.0,4200.75,"Samarqand","2023-06-01"
"Hasan",28.0,3900.0,"Andijon, Toshkent","2022-12-10"
"Husan","",4100.125,"Toshkent",""
```

```
In [79]: # 3. Umuman qo'shtirnoq ishlatmaslik (ba'zi tizimlar uchun)
csv_matn = df.to_csv(None, index=False, quoting=csv.QUOTE_NONE, escapechar='\\')
print(csv_matn)
```

```
name,age,salary,city,join_date
Ali\, Vali,25.0,3500.5,Toshkent,2024-01-15
,30.0,4200.75,Samarqand,2023-06-01
Hasan,28.0,3900.0,Andijon\, Toshkent,2022-12-10
Husan,,4100.125,Toshkent,
```

1 2 date_format – sanalarni matniga o'tkazish formati

Tavsif

- `datetime` ustunlari qanday ko'rinishda CSVga yozilishini belgilaydi.
- Misol: `%Y-%m-%d`, `%d.%m.%Y`, `%Y/%m/%d %H:%M`.

Misollar

```
In [81]: # 1. ISO format (YYYY-MM-DD)
csv_matn = df.to_csv(None, index=False, date_format='%Y-%m-%d')
print(csv_matn)
```

```
name,age,salary,city,join_date
"Ali, Vali",25.0,3500.5,Toshkent,2024-01-15
,30.0,4200.75,Samarqand,2023-06-01
Hasan,28.0,3900.0,"Andijon, Toshkent",2022-12-10
Husan,,4100.125,Toshkent,
```

```
In [82]: # 2. Kun-Oy-Yil formatida
csv_matn = df.to_csv(None, index=False, date_format='%d.%m.%Y')
print(csv_matn)
```

```
name,age,salary,city,join_date
"Ali, Vali",25.0,3500.5,Toshkent,15.01.2024
,30.0,4200.75,Samarqand,01.06.2023
Hasan,28.0,3900.0,"Andijon, Toshkent",10.12.2022
Husan,,4100.125,Toshkent,
```

1 3 mode – faylni yozish rejimi

Tavsif

- `mode='w'` – eski faylni to'liq ustidan yozadi (default).
- `mode='a'` – oxiridan davom ettirib yozadi (append).
- Append rejimida odatda `header=False` qilish kerak bo'ladi.

Misollar

```
In [83]: # 1. Faylni har safar yangidan yozish (default xatti-harakat)
csv_matn = df.to_csv(None, index=False, mode='w')
print(csv_matn)
```

```
name,age,salary,city,join_date
"Ali, Vali",25.0,3500.5,Toshkent,2024-01-15
,30.0,4200.75,Samarqand,2023-06-01
Hasan,28.0,3900.0,"Andijon, Toshkent",2022-12-10
Husan,,4100.125,Toshkent,
```

```
In [88]: # 2. Fayl oxiridan davom ettirish (header=False bo'lmasa, har safar ustun nomlari ham yozilib
csv_matn = df.to_csv(None, index=False, mode='a', header=False)
print(csv_matn)
```

```
"Ali, Vali",25.0,3500.5,Toshkent,2024-01-15
,30.0,4200.75,Samarqand,2023-06-01
Hasan,28.0,3900.0,"Andijon, Toshkent",2022-12-10
Husan,,4100.125,Toshkent,
```

!Bu yerda None o'rniga fayl nomi yoki yo'li ko'rsatilsa, shu faylning davomidan yozadi

1 4 compression – siqilgan faylga yozish (ZIP, GZIP va h.k.)

Tavsif

- Katta CSV fayllar uchun disk joyini tejaydi.
- Keng tarqalgan variantlar: `'zip'`, `'gzip'`, `'bz2'`, `'xz'`.

Misollar

```
# 1. GZIP bilan yozish (.gz kengaytmali)
df.to_csv('employees.csv.gz', index=False, compression='gzip')
# 2. ZIP arxiv ichiga yozish
df.to_csv('employees.zip', index=False, compression='zip')
```

1 5 chunksize – bo'lib-bo'lib (chunk) yozish

Tavsif

- Juda katta DataFramelarni CSVga yozishda xotirani tejash uchun.
- Har bir chaqirishda `chunksize` qatorlik bo'laklar bilan yozadi.

⚠ E'tibor bering: `df.to_csv(chunksize=1000)` – ichkarida avtomatik iteratsiya qiladi, alohida sikl shart emas.

Misollar

```
# Katta DataFrame misoli uchun (tasavvur qiling, df juda katta):  
df.to_csv('employees_big_chunks.csv', index=False, chunksize=1000)
```

Yakuniy umumlashtirilgan misol

Barcha ko'p ishlatiladigan parametrlardan foydalangan holda bitta kompleks misol:

```
import csv  
  
df.to_csv(  
    'employees_final.csv',  
    sep=';',           # ustunlarni ; bilan ajratish  
    index=False,      # indeksni yozmaslik  
    header=True,       # ustun nomlarini yozish  
    columns=['name', 'age', 'salary', 'city', 'join_date'],  
    na_rep='yo'q',     # NaN o'rniga 'yo'q'  
    float_format='%.2f', # maoshni verguldan keyin 2 xona bilan yozish  
    encoding='utf-8-sig', # Excel uchun qulay  
    line_terminator='\n', # satr tugashi  
    quotechar='"',      # matnni qo'shtirnoqqa olish  
    quoting=csv.QUOTE_MINIMAL,  
    date_format='%Y-%m-%d', # sanalar yillik formatda  
    mode='w',           # faylni yangidan yozish  
    # compression='gzip', # kerak bo'lsa, siqib yozish  
)
```

DataFrame.to_excel() bo'yicha to'liq o'quv material

! Muhim eslatma: `to_excel(None, ...)` ishlamaydi

`DataFrame.to_excel()` metodi `None` qabul qilmaydi.

- `to_csv(None)` → matn qaytaradi
- `to_excel(None)` → **xato beradi**

Sabab: Excel fayl **binar format (.xlsx)** bo'lgani uchun Pandas unga **albatta yoziladigan joy (fayl yo'li)** ko'rsatishni talab qiladi.

0 Boshlanishi – DataFrame yaratish

```
In [99]: data = {
    "name": ["Ali", "Vali", "Hasan", "Husan"],
    "age": [25, 30, 28, np.nan],
    "salary": [3500.5, 4200.75, 3900.0, 4100.125],
    "city": ["Toshkent", "Samarqand", "Buxoro", "Toshkent"],
    "join_date": pd.to_datetime(["2024-01-15", "2023-06-01", "2022-12-10", "2024-02-20"])
}

df = pd.DataFrame(data)
df
```

```
Out[99]:
```

	name	age	salary	city	join_date
0	Ali	25.0	3500.500	Toshkent	2024-01-15
1	Vali	30.0	4200.750	Samarqand	2023-06-01
2	Hasan	28.0	3900.000	Buxoro	2022-12-10
3	Husan	NaN	4100.125	Toshkent	2024-02-20

◆ DataFrame.to_excel() umumiy ko'rinishi

```
df.to_excel(
    "fayl.xlsx",          # yoziladigan Excel fayl
    sheet_name='Sheet1',
    index=True,
    header=True,
    columns=None,
    na_rep='',
    float_format=None,
    date_format=None,
    startrow=0,
    startcol=0,
    engine=None,
    freeze_panes=None
)
```

Har bir parametr quyida batafsil tushuntiriladi.

1 excel_writer – yoziladigan Excel fayl

```
df.to_excel("employees.xlsx", index=False)
```

To'liq yo'l bilan:

```
df.to_excel(r"D:\data\employees.xlsx", index=False)
```

2 sheet_name – varaq nomi

```
df.to_excel("employees_sheet.xlsx", index=False, sheet_name="Xodimlar")
```

sheet_name ko'rsatilmasa — default Sheet1 ga yozadi

DataFrame.to_excel() chaqirilganda agar sheet_name parametri yozilmasa, Pandas avtomatik ravishda:

- Excel fayl ichida **bitta varaq yaratadi**
- uning nomini **"Sheet1"** deb belgilaydi

Misol:

```
df.to_excel("employees.xlsx", index=False)
```

Natija:

- employees.xlsx fayli yaratiladi
- ichida **faqat bitta sheet bo'ladi**
- varaq nomi: **Sheet1**

Bu Microsoft Excel'ning standart nomlash tartibiga mos keladi.

3 index – indeksni yozish/yozmaslik

Indekssiz:

```
df.to_excel("employees_no_index.xlsx", index=False)
```

Indeks bilan (default):

```
df.to_excel("employees_with_index.xlsx", index=True)
```

4 header – ustun nomlarini yozish/yozmaslik

Ustun nomlarini o'chirish:

```
df.to_excel("employees_no_header.xlsx", index=False, header=False)
```

Custom header:

```
df.to_excel(  
    "employees_custom_header.xlsx",  
    index=False,  
    header=["Ism", "Yosh", "Maosh", "Shahar", "Sana"]  
)
```

5 columns – faqat tanlangan ustunlar

```
df.to_excel(  
    "employees_name_salary.xlsx",  
    index=False,  
    columns=["name", "salary"]  
)
```

6 na_rep – NaN qiymat o'rniga yozish

```
df.to_excel("employees_na.xlsx", index=False, na_rep="yo'q")
```

7 float_format – haqiqiy sonni formatlash

verguldan keyin 2 ta son bilan:

```
df.to_excel("employees_float2.xlsx", index=False, float_format="%.2f")
```

verguldan keyin 3 ta son bilan:

```
df.to_excel("employees_float3.xlsx", index=False, float_format="%.3f")
```

8 date_format – sana formatlash

```
df.to_excel("employees_date_iso.xlsx", index=False, date_format="%Y-%m-%d")
```

Yoki:

```
df.to_excel("employees_date_dmy.xlsx", index=False, date_format="%d.%m.%Y")
```

9 startrow va startcol – jadvalning joyi

4-qator, C ustundan:

```
df.to_excel(  
    "employees_start_pos.xlsx",  
    index=False,
```



```
startrow=3,  
startcol=2  
)
```

10. freeze_panes – qator/ustunni muzlatish

Bir qatorni muzlatish (header qatori):

```
df.to_excel(  
    "employees_freeze_header.xlsx",  
    index=False,  
    freeze_panes=(1, 0)  
)
```

Birinchi qator + birinchi ustunni muzlatish:

```
df.to_excel(  
    "employees_freeze_row_col.xlsx",  
    index=False,  
    freeze_panes=(1, 1)  
)
```

freeze_panes — bu **jadvalda yuqoridagi qatorlar yoki chapdagi ustunlarni muzlatib qo'yish** imkonini beradigan parametr.

Nega muzlatish kerak?

Excelda katta hajmdagi jadvalni ko'rayotganda:

- pastga skroll qilganda **ustun sarlavhalari yo'qolib ketadi**
- o'ngga skroll qilganda **qator nomlari yo'qolib ketadi**

Shu sababli:

Muzlatishning asosiy maqsadlari

1. Ustun sarlavhalarini ko'rinib turishi

Katta jadvalda 100 qator bo'lsa, 80-90-qatorlarda ko'rayotganda sarlavhalarni eslab bo'lmaydi. Shuning uchun birinchi qatorni muzlatish foydali:

```
freeze_panes=(1, 0)
```

Bu — **header doim tepada ko'rinib turadi**.

2. Qator nomlarini yo'qotmaslik

Jadvalda chapdan o'ngga ko'p ustunlar bo'lsa, skroll qilinganda qaysi qatorda ekaningizni bilish qiyin. Shuning uchun birinchi ustunni muzlatish qulay:

```
freeze_panes=(0, 1)
```

Bu — **qator identifikatori har doim chapda ko'rinadi**.

3. Ham qator, ham ustunni birga muzlatish

Eng ko'p ishlatiladigan variant:

```
freeze_panes=(1, 1)
```

Bu:

- yuqoridagi sarlavha
- chapdagi identifikator

ikkovini ham skrollda yo'qolib ketishdan himoya qiladi.

Qisqa qilib aytganda:

freeze_panes — jadvalni o'qishni osonlashtirish uchun, skrollda yo'qolib ketadigan muhim qator/ustunlarni doim ko'rinib turishini ta'minlaydi.

Ayniqsa:

- ko'p ustunli maosh jadvali
- katta statistik ma'lumotlar
- talabalar ro'yxati
- inventarizatsiya ro'yxatlari

kabi jadvallarda juda kerak.

11. engine – backend tanlash

Openpyxl:

```
df.to_excel("employees_openpyxl.xlsx", index=False, engine="openpyxl")
```

XlsxWriter:

```
df.to_excel("employees_xlsxwriter.xlsx", index=False, engine="xlsxwriter")
```

`engine` parametri — **Excel faylini qanday kutubxona orqali yozishni tanlash** uchun kerak bo'ladi.

Pandas o'zi Excel yozishni qilmaydi, balki boshqa kutubxonalar yordamida bajaradi. Shu kutubxonani siz `engine=` orqali tanlaysiz.

Nega engine kerak?

Excel `.xlsx` faylini yaratish uchun Pandas quyidagi kutubxonalardan foydalanadi:

- **openpyxl** → eng keng tarqalgan va default
- **xlsxwriter** → kuchli formatlash imkoniyatlari
- **odfpy** → `.ods` fayllar uchun
- **pyxlsb** → `.xlsb` (binary workbook) uchun

Agar siz `engine=` parametrini ko'rsatmasangiz, Pandas:

- `.xlsx` uchun **openpyxl**
- `.xls` uchun **xlwt**
- `.ods` uchun **odfpy**

kutubxonasidan foydalanadi.

1. openpyxl — default `xlsx` yozuvchi

Nima uchun ishlatiladi:

- `.xlsx` fayllar uchun standart
- oson ishlaydi, umumiy vazifalar uchun yetarli
- boshqa kutubxonalar bilan yaxshi moslashadi

Qachon tanlanadi:

- oddiy jadval yozishda
- formatlash talab qilinmaganda

2. xlsxwriter — kuchli formatlash uchun

Xususiyatlari:

- murakkab formatlash (ranglar, borderlar, merge)
- diagramma (chart) qo'shish
- shartli formatlash (conditional formatting)
- ustun kengligini boshqarish
- `freeze_panes`, `autofilter`, `set_column` kabi kuchli metodlar

Qachon kerak bo'ladi:

- Agar siz Excel faylni dizayn qilmoqchi bo'lsangiz
- Formatlash ustun bo'lsa
- Jadvalni prezentatsiya uchun chiroyli ko'rinishda tayyorlash kerak bo'lsa

Qisqa taqqoslash:

Engine	Kuchli tomonlari	Qachon ishlatiladi
openpyxl	Oddiy, barqaror, default	Ko'p hollarda
xlsxwriter	Kuchli formatlash, diagramma	Rasmiy fayl dizaynlari uchun
odfpy	.ods Format	LibreOffice
pyxlsb	.xlsb Format	Binary workbook kerak bo'lsa

■ Juda qisqa xulosa:

`engine=` → **Excel faylini qaysi kutubxona yozishini belgilaydi**. Agar formatlash zarur bo'lsa — `xlsxwriter`. Oddiy fayl uchun — default `openpyxl` yetarli.

12. Bir nechta varaqni bitta faylga yozish

```
with pd.ExcelWriter("employees_multi.xlsx", engine="openpyxl") as writer:
    df.to_excel(writer, sheet_name="Asosiy", index=False)
    df[["name", "salary"]].to_excel(writer, sheet_name="Maosh", index=False)
    df["city"].drop_duplicates().to_excel(writer, sheet_name="Shaharlar",
index=False)
```

Quyidagi kodda **bitta Excel fayl ichiga bir nechta sheet (varaq)** yozilmoqda. Pandasda bir nechta varaq yaratish uchun `ExcelWriter` ishlatiladi.

```
with pd.ExcelWriter("employees_multi.xlsx", engine="openpyxl") as writer:
    df.to_excel(writer, sheet_name="Asosiy", index=False)
    df[["name", "salary"]].to_excel(writer, sheet_name="Maosh", index=False)
    df["city"].drop_duplicates().to_excel(writer, sheet_name="Shaharlar",
index=False)
```

■ Qadamlar bo'yicha tushuntirish

1. `ExcelWriter` — Excel faylini boshqaruvchi obyekt

```
with pd.ExcelWriter("employees_multi.xlsx", engine="openpyxl") as writer:
```

Bu qator:

- `employees_multi.xlsx` faylini yaratadi
- yozish jarayonini boshqarish uchun **writer** obyektini ochadi
- `with` blok tugaganda fayl avtomatik **saqlanadi va yopiladi**

2. Birinchi varaq — to'liq DataFrame

```
df.to_excel(writer, sheet_name="Asosiy", index=False)
```

- Fayl ichida **Asosiy** nomli varaq yaratiladi
- Barcha DataFrame shu varaqqa yoziladi

3. Ikkinchi varaq — faqat 2 ta ustun

```
df[["name", "salary"]].to_excel(writer, sheet_name="Maosh", index=False)
```

- **Maosh** nomli varaq yaratiladi
- Unda faqat `name` va `salary` ustunlari yoziladi

4. Uchinchi varaq — shaharlar ro'yxati (takrorlarsiz)

```
df["city"].drop_duplicates().to_excel(writer, sheet_name="Shaharlar", index=False)
```

- **Shaharlar** nomli varaq yaratiladi
 - Unda takrorlanmaydigan shaharlar ro'yxati yoziladi (masalan: Toshkent, Samarqand, Buxoro)
-

Natija

employees_multi.xlsx fayli quyidagi 3 ta sheetga ega bo'ladi:

1. **Asosiy** — to'liq jadval
 2. **Maosh** — faqat ism va maosh
 3. **Shaharlar** — takrorlarsiz shahar nomlari
-



Yakuniy – kompleks parametrlil misol

```
df.to_excel(  
    "employees_final.xlsx",  
    sheet_name="Xodimlar",  
    index=False,  
    header=True,  
    columns=["name", "age", "salary", "city", "join_date"],  
    na_rep="yo'q",  
    float_format="%.2f",  
    date_format="%Y-%m-%d",  
    startrow=1,  
    startcol=1,  
    freeze_panes=(2, 1)  
)
```

4. Ma'lumotlarni O'zgartirish

Yangi ustunlar qo'shish

Pandasda yangi ustun qo'shish juda oson. Siz yangi ustunlarni DataFrame'ga turli usullar bilan qo'shishingiz mumkin.

Yangi ustun qo'shish

1. **Qimmatli ma'lumotlar asosida yangi ustun qo'shish:**

```
import pandas as pd  
  
data = {  
    'Ism': ['Ali', 'Veli', 'Sami'],  
    'Yosh': [23, 45, 34],  
    'Shahar': ['Tashkent', 'Samarkand', 'Bukhara']  
}  
  
df = pd.DataFrame(data)
```

```
# Yangi ustun qo'shish
df['Yoshdan_kattaroq'] = df['Yosh'] > 30
print(df)
```

Natija:

	Ism	Yosh	Shahar	Yoshdan_kattaroq
0	Ali	23	Tashkent	False
1	Veli	45	Samarkand	True
2	Sami	34	Bukhara	True

Bu yerda **Yoshdan_kattaroq** ustuni **Yosh** ustunidagi qiymatlarga qarab haqiqiy (True) yoki yolg'on (False) qiymatlarni saqlaydi.

2. Funksiya yordamida yangi ustun qo'shish:

```
# Yangi ustun yaratish va funksiyadan foydalanish
df['Yosh_2x'] = df['Yosh'].apply(lambda x: x * 2)
print(df)
```

Natija:

	Ism	Yosh	Shahar	Yoshdan_kattaroq	Yosh_2x
0	Ali	23	Tashkent	False	46
1	Veli	45	Samarkand	True	90
2	Sami	34	Bukhara	True	68

apply() metodi yordamida **Yosh** ustunining har bir qiymatini ikki baravarga oshirdik.

Ma'lumotlarni Tozalash

Pandasda **tozalash** — bu ma'lumotlarda bo'sh yoki noto'g'ri qiymatlar mavjud bo'lsa, ularni tuzatish yoki olib tashlash jarayonidir.

Bo'sh yoki NaN qiymatlarni aniqlash va olib tashlash

1. NaN qiymatlarni tekshirish:

Bo'sh qiymatlar (NaN) mavjudligini **isnull()** yordamida tekshirish mumkin:

```
df = pd.DataFrame({
    'Ism': ['Ali', 'Veli', 'Sami'],
    'Yosh': [23, None, 34],
    'Shahar': ['Tashkent', 'Samarkand', None]
})

print(df.isnull())
```

Natija:

	Ism	Yosh	Shahar
0	False	False	False
1	False	True	False
2	False	False	True

2. NaN qiymatlarni olib tashlash:

dropna() metodi yordamida **NaN** qiymatlari mavjud bo'lgan qator yoki ustunlarni olib tashlash mumkin:

- **Qatorlarni olib tashlash:**

```
df_clean = df.dropna(axis=0) # NaN bo'lgan qatorlarni olib tashlash
print(df_clean)
```

Natija:

	Ism	Yosh	Shahar
0	Ali	23	Tashkent

- **Ustunlarni olib tashlash:**

```
df_clean = df.dropna(axis=1) # NaN bo'lgan ustunlarni olib tashlash
print(df_clean)
```

Natija:

	Ism	Yosh
0	Ali	23
1	Veli	None
2	Sami	34

3. NaN qiymatlarni to'ldirish:

`fillna()` metodi yordamida `NaN` qiymatlarni belgilangan qiymat bilan to'ldirish mumkin:

```
df_filled = df.fillna(value={'Yosh': 30, 'Shahar': 'Noma'})
```

```
print(df_filled)
```

Natija:

	Ism	Yosh	Shahar
0	Ali	23	Tashkent
1	Veli	30	Samarkand
2	Sami	34	Noma

Dublikatlarni olib tashlash

Pandasda dublikatlarni olib tashlash uchun `drop_duplicates()` metodi ishlatiladi. Bu metod bir xil qiymatlarga ega bo'lgan qatorlarni olib tashlashga yordam beradi.

```
df = pd.DataFrame({
    'Ism': ['Ali', 'Veli', 'Ali'],
    'Yosh': [23, 45, 23],
    'Shahar': ['Tashkent', 'Samarkand', 'Tashkent']
})
```

```
df_clean = df.drop_duplicates()
print(df_clean)
```

Natija:

	Ism	Yosh	Shahar
0	Ali	23	Tashkent
1	Veli	45	Samarkand

Bu yerda **Ali** qatori ikkinchi marta takrorlangan, shuning uchun uni olib tashladik.

Ma'lumot Turlarini O'zgartirish

Pandasda ma'lumotlar turini o'zgartirish uchun `astype()` metodidan foydalanish mumkin. Bu usul ma'lumotlar turini kerakli formatga o'zgartiradi.

1. Ma'lumot turini o'zgartirish:

```
df['Yosh'] = df['Yosh'].astype(float)
print(df.dtypes)
```

Natija:

```
Ism          object
Yosh         float64
Shahar       object
dtype: object
```

2. Bir nechta ustunlarni turini o'zgartirish:

Bir nechta ustunlarning turini o'zgartirish uchun `astype()` metodini dictionary yordamida ishlatish mumkin:

```
df = df.astype({'Yosh': 'float64', 'Shahar': 'category'})
print(df.dtypes)
```

Natija:

```
Ism          object
Yosh         float64
Shahar       category
dtype: object
```

Xulosa

- **Yangi ustunlar qo'shish:** Pandasda yangi ustunlar yaratish va mavjud ustunlarga turli matematik yoki funksional operatsiyalarni qo'llash mumkin.
- **Ma'lumotlarni tozalash:** Pandas yordamida ma'lumotlardagi **NaN** qiymatlarni aniqlash, olib tashlash yoki to'ldirish imkoniyatlari mavjud.
- **Dublikatlarni olib tashlash:** Pandasda `drop_duplicates()` metodi yordamida takrorlangan ma'lumotlarni olib tashlash mumkin.
- **Ma'lumot turlarini o'zgartirish:** `astype()` metodi yordamida ma'lumot turlarini kerakli formatga o'zgartirish mumkin.

5. Ma'lumotlarni Tahlil Qilish

Deskriptiv Statistikalalar

Deskriptiv statistika — bu ma'lumotlarni umumiy tahlil qilish va ularning asosiy xususiyatlarini aniqlash uchun ishlatiladigan statistika turi. Pandas kutubxonasida deskriptiv statistikalarni olish uchun bir nechta foydali metodlar mavjud.

O'rtacha, maksimal, minimal va boshqalar

1. O'rtacha qiymat (`mean`):

O'rtacha qiymatni hisoblash uchun `mean()` metodidan foydalanamiz:


```
df = pd.DataFrame({
    'Ism': ['Ali', 'Veli', 'Sami'],
    'Yosh': [23, 45, 34]
})
```

```
mean_yosh = df['Yosh'].mean()
print("Oʻrtacha yosh:", mean_yosh)
```

Natija:

Oʻrtacha yosh: 34.0

2. **Maksimal va minimal qiymatlar** (`max` , `min`):

- Maksimal qiymatni olish uchun `max()` :

```
max_yosh = df['Yosh'].max()
print("Maksimal yosh:", max_yosh)
```

- Minimal qiymatni olish uchun `min()` :

```
min_yosh = df['Yosh'].min()
print("Minimal yosh:", min_yosh)
```

Natija:

Maksimal yosh: 45

Minimal yosh: 23

3. **Standart ogʻish** (`std`):

`std()` metodidan foydalanib, ma'lumotlarning tarqalishini (standard deviation) hisoblash mumkin:

```
std_yosh = df['Yosh'].std()
print("Standart ogʻish:", std_yosh)
```

Natija:

Standart ogʻish: 11.0

4. **Mediana** (`median`):

`median()` metodi yordamida oʻrtadagi qiymatni aniqlash mumkin:

```
median_yosh = df['Yosh'].median()
print("Mediana:", median_yosh)
```

Natija:

Mediana: 34.0

5. **Moda** (`mode`):

`mode()` metodi yordamida eng koʻp uchraydigan qiymatni aniqlash mumkin:

```
mode_yosh = df['Yosh'].mode()
print("Moda:", mode_yosh)
```

Natija:

Moda: 23 23

dtype: int64

6. **Ma'lumotlar haqida umumiy statistikalar** (`describe`):

`describe()` metodi yordamida ma'lumotlar to'plamining umumiy statistikalarini olish mumkin, masalan, oʻrtacha, minimum, maksimum, kvartil va boshqalar:

```
print(df.describe())
```

Natija:

	Yosh
count	3.000000
mean	34.000000
std	11.000000
min	23.000000
25%	28.500000
50%	34.000000
75%	39.500000
max	45.000000

groupby() va Agregatsiyalar yordamida Ma'lumotlarni Guruhlash

Pandasda ma'lumotlarni guruhlash va agregatsiya qilish uchun `groupby()` metodi ishlatiladi. Bu metod ma'lumotlarni o'xshash xususiyatlarga asoslanib guruhlaydi va har bir guruh uchun agregatsiya funksiyalarini qo'llash imkonini beradi.

groupby() metodini ishlatish

1. Guruhlash va o'rtacha qiymatni olish:

Faraz qilaylik, bizda quyidagi ma'lumotlar to'plami bor:

```
df = pd.DataFrame({
    'Ism': ['Ali', 'Veli', 'Sami', 'Zafar', 'Sara'],
    'Shahar': ['Tashkent', 'Samarkand', 'Tashkent', 'Samarkand', 'Tashkent'],
    'Yosh': [23, 45, 34, 34, 28]
})

# Shahar bo'yicha guruhlash va o'rtacha yoshni hisoblash
df_grouped = df.groupby('Shahar')['Yosh'].mean()
print(df_grouped)
```

Natija:

Shahar	
Samarkand	39.5
Tashkent	28.333333

Name: Yosh, dtype: float64

Bu yerda **Shahar** bo'yicha guruhlab, har bir shahar uchun o'rtacha yoshni hisobladik.

2. Guruhlash va boshqa agregatsiya funksiyalarini qo'llash:

`groupby()` metodiga turli agregatsiya funksiyalarini qo'llash mumkin, masalan, **sum**, **min**, **max** va boshqalar:

```
df_grouped = df.groupby('Shahar')['Yosh'].agg([min, max, sum, 'mean'])
print(df_grouped)
```

Natija:

	min	max	sum	mean
Shahar				
Samarkand	34	45	69	39.5
Tashkent	23	34	85	28.333333

Bu yerda **Samarkand** va **Tashkent** bo'yicha minimal, maksimal, yig'indi va o'rtacha yoshlarni hisobladik.

3. Bir nechta ustun bo'yicha guruhlash:

Agar bir nechta ustun bo'yicha guruhlashni xohlasangiz, buni quyidagi tarzda amalga oshirish mumkin:

```
df_grouped = df.groupby(['Shahar', 'Ism'])['Yosh'].mean()
print(df_grouped)
```

Natija:

Shahar	Ism	
Samarkand	Veli	45.0
	Zafar	34.0
Tashkent	Ali	23.0
	Sami	34.0
	Sara	28.0

Name: Yosh, dtype: float64

Bu yerda **Shahar** va **Ism** bo'yicha guruhlab, har bir shaxsning o'rtacha yoshini hisobladik.

4. Pivot jadval yaratish:

Pivot jadval yordamida ma'lumotlarni guruhlash va agregatsiya qilishni amalga oshirishingiz mumkin:

```
df_pivot = df.pivot_table(values='Yosh', index='Shahar', aggfunc='mean')
print(df_pivot)
```

Natija:

	Yosh
Shahar	
Samarkand	39.5
Tashkent	28.333333

Xulosa

- **Deskriptiv statistika:** Pandas yordamida o'rtacha, maksimal, minimal, mediana, moda va boshqa statistiklarni hisoblash juda oson. `describe()` metodi ma'lumotlar to'plamining umumiy statistikalarini taqdim etadi.
- **groupby() va agregatsiya:** Ma'lumotlarni guruhlash va har bir guruh uchun agregatsiya funksiyalarini qo'llash mumkin. Bu, ayniqsa, katta ma'lumotlar to'plamlarini tahlil qilishda foydalidir.

6. Ma'lumotlarni Vizualizatsiya Qilish

Ma'lumotlarni vizualizatsiya qilish — bu tahlil jarayonining muhim qismidir, chunki u ma'lumotlar bilan bog'liq o'rganilayotgan xususiyatlarni tezda tushunishga yordam beradi. Pandas, **Matplotlib**, va **Seaborn** kutubxonalari yordamida ma'lumotlarni turli grafikalar yordamida ko'rsatish mumkin.

Pandas va Matplotlib yordamida Grafiklar Yaratish

Pandas kutubxonasi, o'zining `plot()` metodini ishlatib, ma'lumotlarni vizualizatsiya qilish imkonini beradi. Pandas DataFrame va Series ob'ektlari Matplotlibni ichki kutubxona sifatida ishlatadi.

Matplotlibni o'rnatish

Agar Matplotlib kutubxonasi o'rnatilmagan bo'lsa, quyidagicha o'rnatishingiz mumkin:

```
pip install matplotlib
```

Pandas yordamida oddiy chizmalar yaratish

Pandas yordamida ma'lumotlar to'plamidan turli xil grafiklarni yaratish mumkin. Misol uchun:

1. Chiziqli grafik (Line Plot):

```
import pandas as pd
import matplotlib.pyplot as plt

# Ma'lumotlarni yaratish
df = pd.DataFrame({
    'Yil': [2020, 2021, 2022, 2023, 2024],
    'Savdo': [120, 150, 170, 160, 180]
})

# Chiziqli grafikni yaratish
df.plot(x='Yil', y='Savdo', kind='line', marker='o', linestyle='-', color='b')

# Grafikni ko'rsatish
plt.title('Yillik Savdo')
plt.xlabel('Yil')
plt.ylabel('Savdo')
plt.grid(True)
plt.show()
```

Bu chizma **Savdo** va **Yil** o'rtasidagi o'zgarishni ko'rsatadi.

2. Gistogramma (Histogram):

```
df = pd.DataFrame({
    'Yosh': [23, 45, 34, 23, 45, 50, 35, 29, 30, 40]
})

# Gistogramma yaratish
df['Yosh'].plot(kind='hist', bins=5, color='g', alpha=0.7)

plt.title('Yosh Taqsimoti')
plt.xlabel('Yosh')
plt.ylabel('Chastota')
plt.show()
```

Gistogramma yordamida ma'lumotlar taqsimotini ko'rish mumkin.

3. Scatter plot (Tarqalish diagrammasi):

```
df = pd.DataFrame({
    'X': [1, 2, 3, 4, 5],
    'Y': [2, 4, 5, 4, 5]
})

# Scatter plot yaratish
df.plot(kind='scatter', x='X', y='Y', color='r', marker='x')

plt.title('X va Y o'rtasidagi munosabat')
```

```
plt.xlabel('X')
plt.ylabel('Y')
plt.show()
```

Scatter plot ikki o'zgaruvchining o'rtasidagi munosabatni ko'rsatadi.

Matplotlib bilan Pandas grafikasini sozlash

Matplotlib yordamida chizmalar va grafiklarni sozlash mumkin. Quyidagi misolda grafikni sozlash ko'rsatilgan:

```
df.plot(x='Yil', y='Savdo', kind='bar', color='cyan', edgecolor='black')

plt.title('Savdo Bo'yicha Yillik Hisobot')
plt.xlabel('Yil')
plt.ylabel('Savdo (mln $)')
plt.show()
```

Seaborn bilan Integratsiya

Seaborn — bu Matplotlib ustida qurilgan va ma'lumotlarni tahlil qilishda qulayroq va estetik jihatdan chiroyli vizualizatsiyalarni yaratish uchun ishlatiladigan kutubxona. Seaborn ko'plab statistik grafikalar yaratishga yordam beradi.

Seabornni o'rnatish

Agar Seaborn o'rnatilmagan bo'lsa, quyidagicha o'rnatishingiz mumkin:

```
pip install seaborn
```

Seaborn yordamida grafikalar yaratish

Seaborn yordamida oddiy va murakkab grafiklarni yaratish mumkin.

1. Boxplot (Quti diagrammasi):

Boxplot ma'lumotlar taqsimotini va ular orasidagi farqlarni ko'rsatish uchun ishlatiladi.

```
import seaborn as sns
```

```
df = pd.DataFrame({
    'Yosh': [23, 45, 34, 23, 45, 50, 35, 29, 30, 40],
    'Shahar': ['Tashkent', 'Samarkand', 'Bukhara', 'Tashkent', 'Samarkand',
               'Bukhara', 'Tashkent', 'Samarkand', 'Bukhara', 'Tashkent']
})
```

```
sns.boxplot(x='Shahar', y='Yosh', data=df)
plt.title('Yoshning Shaharlar Bo'yicha Taqqoslanishi')
plt.show()
```

Bu **boxplot** yosh taqsimotini shaharlarga qarab ko'rsatadi.

2. Pairplot (Juftlik diagrammasi):

Pairplot yordamida bir nechta o'zgaruvchilar o'rtasidagi o'zaro munosabatni ko'rish mumkin.

```
df = pd.DataFrame({
    'X': [1, 2, 3, 4, 5],
    'Y': [2, 4, 5, 4, 5],
})
```

```
    'Z': [5, 7, 8, 6, 7]  
})
```

```
sns.pairplot(df)  
plt.show()
```

3. Heatmap (Isitish xaritasi):

Heatmap yordamida ma'lumotlar orasidagi korrelyatsiyani vizual tarzda ko'rsatish mumkin.

```
correlation_matrix = df.corr()
```

```
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')  
plt.title('Ma'lumotlar orasidagi Korrelyatsiya')  
plt.show()
```

Bu **heatmap** ma'lumotlar o'rtasidagi korrelyatsiyani ko'rsatadi.

Seaborn va Pandas integratsiyasi

Pandas va Seaborn birgalikda ishlash uchun juda qulay. Misol uchun, Pandas DataFrame'ni Seaborn yordamida vizualizatsiya qilish:

```
sns.barplot(x='Yil', y='Savdo', data=df, palette='Blues_d')  
plt.title('Yillik Savdo')  
plt.show()
```

Bu misolda Pandas DataFrame'ni Seaborn yordamida **bar plot** shaklida ko'rsatdik.

Xulosa

- **Pandas va Matplotlib yordamida grafiklar yaratish:** Pandas kutubxonasi yordamida ma'lumotlarni o'qib, ular bilan ishlash va Matplotlib yordamida turli grafikalar yaratish mumkin.
- **Seaborn bilan integratsiya:** Seaborn kutubxonasi, statistik grafikalar yaratishda, ayniqsa, estetika va o'qilishi oson grafikalar yaratishda juda foydalidir. Seaborn va Pandas birgalikda ishlash orqali ma'lumotlarni yanada mukammal tarzda vizualizatsiya qilish mumkin.

Bu imkoniyatlar yordamida siz ma'lumotlaringizni to'g'ri va samarali tahlil qilishingiz va ularni vizual ravishda ifodalashingiz mumkin.

7. Ilg'or Pandas Imkoniyatlari

Pandas kutubxonasi, nafaqat oddiy ma'lumotlar bilan ishlash, balki **vaqt bo'yicha ma'lumotlar** va **ma'lumotlarni birlashtirish** kabi ilg'or funksiyalarni ham taqdim etadi. Ushbu imkoniyatlar yordamida ma'lumotlarni yanada samarali va kengaytirilgan tarzda tahlil qilish mumkin.

Vaqt Bo'yicha Ma'lumotlar Bilan Ishlash

Pandasda vaqt bo'yicha ma'lumotlar bilan ishlash uchun **datetime** kutubxonasi va `to_datetime()` metodidan foydalaniladi. Bu imkoniyat, ayniqsa, vaqtga asoslangan tahlil va prognozlash uchun juda foydalidir.

Vaqtga asoslangan indekslash

1. Vaqtga asoslangan indeks yaratish:

```
import pandas as pd

# Vaqtli indeksli DataFrame yaratish
date_range = pd.date_range(start='2021-01-01', periods=6, freq='D')
df = pd.DataFrame({
    'Date': date_range,
    'Temperature': [22, 21, 23, 24, 25, 26]
})

df.set_index('Date', inplace=True)
print(df)
```

Natija:

Date	Temperature
2021-01-01	22
2021-01-02	21
2021-01-03	23
2021-01-04	24
2021-01-05	25
2021-01-06	26

Bu yerda `date_range()` yordamida 6 kunlik vaqtli indeks yaratildi.

2. Vaqt bo'yicha filtrlar:

Vaqtli indekslar yordamida oson filtrlar yaratish mumkin. Masalan, ma'lumotlarni ma'lum bir sanadan oldin yoki keyin ajratish:

```
# 2021-01-04 dan keyingi ma'lumotlarni olish
df_filtered = df[df.index >= '2021-01-04']
print(df_filtered)
```

Natija:

Date	Temperature
2021-01-04	24
2021-01-05	25
2021-01-06	26

3. Vaqtga asoslangan agregatsiya:

Vaqtli indekslar yordamida ma'lumotlarni o'rnatilgan vaqt intervaliga qarab agregatsiya qilish mumkin, masalan, kunlik, oylik yoki yillik agregatsiya.

```
# Kunlik o'rtacha qiymatni hisoblash
daily_avg = df.resample('D').mean()
print(daily_avg)
```

Natija:

Date	Temperature
2021-01-01	22
2021-01-02	21
2021-01-03	23
2021-01-04	24
2021-01-05	25
2021-01-06	26

`resample()` metodi yordamida vaqtli indeks bo'yicha ma'lumotlarni turli o'lchamlarda agregatsiya qilish mumkin (kunlik `D`, oylik `M`, yillik `A` va h.k.).

4. Vaqt bo'yicha farqlarni hisoblash:

Agar ma'lumotlar ketma-ketligi vaqtga asoslangan bo'lsa, farqlarni hisoblash mumkin. Misol uchun, kunlik o'zgarishlarni hisoblash:

```
df['Temperature_change'] = df['Temperature'].diff()  
print(df)
```

Natija:

Date	Temperature	Temperature_change
2021-01-01	22	NaN
2021-01-02	21	-1.0
2021-01-03	23	2.0
2021-01-04	24	1.0
2021-01-05	25	1.0
2021-01-06	26	1.0

`diff()` metodi yordamida vaqtli ketma-ketlikdagi o'zgarishlarni hisoblash mumkin.

Ma'lumotlarni Birlashtirish va Shakllantirish

Pandasda ma'lumotlarni birlashtirish va shakllantirish uchun `merge()`, `concat()`, va `join()` kabi metodlar mavjud. Bu metodlar ma'lumotlarni birlashtirish, bir nechta ma'lumotlar to'plamini birlashtirish va shakllantirishda juda foydalidir.

`merge()` yordamida ma'lumotlarni birlashtirish

1. Ikkita DataFrame'ni birlashtirish:

```
df1 = pd.DataFrame({  
    'ID': [1, 2, 3],  
    'Ism': ['Ali', 'Veli', 'Sami']  
})  
  
df2 = pd.DataFrame({  
    'ID': [1, 2, 4],  
    'Yosh': [23, 45, 34]  
})  
  
# 'ID' ustuni bo'yicha birlashtirish  
df_merged = pd.merge(df1, df2, on='ID', how='inner')  
print(df_merged)
```

Natija:

	ID	Ism	Yosh
0	1	Ali	23
1	2	Veli	45

Bu yerda `merge()` metodi ikkita DataFrame'ni **ID** ustuni bo'yicha birlashtiradi. `how='inner'` – faqat ikkala DataFrame'da mavjud bo'lgan ID'lar bo'yicha birlashtirishni anglatadi. Boshqa turdagi birlashtirishlar uchun `outer`, `left`, `right` parametrlarini ishlatish mumkin.

`concat()` yordamida ma'lumotlarni birlashtirish

concat() metodi bir nechta DataFrame'ni vertikal yoki gorizontal ravishda birlashtirishga yordam beradi:

```
df1 = pd.DataFrame({
    'Ism': ['Ali', 'Veli'],
    'Yosh': [23, 45]
})

df2 = pd.DataFrame({
    'Ism': ['Sami', 'Zafar'],
    'Yosh': [34, 30]
})

# DataFrame'larni vertikal ravishda birlashtirish
df_concatenated = pd.concat([df1, df2], ignore_index=True)
print(df_concatenated)
```

Natija:

	Ism	Yosh
0	Ali	23
1	Veli	45
2	Sami	34
3	Zafar	30

ignore_index=True parametri birlashtirilgan DataFrame'da yangi indekslarni yaratadi.

join() yordamida ma'lumotlarni birlashtirish

join() metodi asosan ikkita DataFrame'ni indeks orqali birlashtirishda ishlatiladi:

```
df1 = pd.DataFrame({
    'Ism': ['Ali', 'Veli'],
    'Yosh': [23, 45]
}, index=['A', 'B'])

df2 = pd.DataFrame({
    'Shahar': ['Tashkent', 'Samarkand']
}, index=['A', 'B'])

# Indeks bo'yicha birlashtirish
df_joined = df1.join(df2)
print(df_joined)
```

Natija:

	Ism	Yosh	Shahar
A	Ali	23	Tashkent
B	Veli	45	Samarkand

join() metodi yordamida ma'lumotlar birlashtiriladi, bu yerda ikkita DataFrame indeksleri bo'yicha birlashtirildi.

Xulosa

- **Vaqt bo'yicha ma'lumotlar:** Pandasda vaqtli indekslar yordamida ma'lumotlarni tahlil qilish va agregatsiya qilish oson. `resample()` yordamida vaqt bo'yicha guruhlash va agregatsiya qilish mumkin.
- **Ma'lumotlarni birlashtirish va shakllantirish:** Pandasda ma'lumotlarni birlashtirish uchun `merge()`, `concat()`, va `join()` metodlaridan foydalanish mumkin. Bu metodlar ma'lumotlarni vertikal va gorizontal ravishda birlashtirish, hamda indekslar bo'yicha birlashtirish imkoniyatini taqdim etadi.

8. Pandasda Optimizatsiya va Samaradorlik

Pandas kutubxonasi katta hajmdagi ma'lumotlar bilan ishlashda samaradorlikni oshirish uchun bir qator optimizatsiya usullarini taqdim etadi. Bu usullar ma'lumotlarni tezroq tahlil qilish, ko'proq xotira tejash va hisoblash jarayonini tezlashtirish uchun foydalidir.

Katta Hajmdagi Ma'lumotlar Bilan Ishlash

Katta hajmdagi ma'lumotlar bilan ishlashda samaradorlikni oshirish uchun ba'zi texnikalar mavjud:

`chunksize` yordamida katta fayllarni bo'lib o'qish

Agar CSV, Excel yoki boshqa katta fayllarni o'qiyotgan bo'lsangiz, butun faylni xotiraga yuklash o'rniga uni bo'lib o'qish mumkin. `chunksize` parametri faylni kichik bo'laklarga ajratib o'qishga yordam beradi.

Misol uchun, katta CSV faylini bo'lib o'qish:

```
import pandas as pd

# Katta faylni bo'lib o'qish (10000 qatorli bo'laklar)
chunk_size = 10000
chunks = pd.read_csv('large_data.csv', chunksize=chunk_size)
```

```
# Har bir bo'lakni alohida ishlash
for chunk in chunks:
    # Ma'lumotlar bilan ishlash
    print(chunk.head()) # Har bir bo'lakdan birinchi 5 qatorni chiqarish
```

Bu usul yordamida siz ma'lumotlarni to'liq o'qib bo'lmasdan turib, bo'laklar bilan ishlay olasiz.

`dtypes` ni optimallashtirish

Agar siz ma'lumotlar turini **optimal** tarzda belgilasangiz, xotira sarfini kamaytirishingiz mumkin. Masalan, integer ustunlarini `int32` yoki `int8` kabi kichikroq turlarga o'zgartirish orqali xotirani tejash mumkin.

```
import pandas as pd

# Katta CSV faylini o'qish
df = pd.read_csv('large_data.csv', dtype={'column1': 'int32', 'column2': 'float32'})

print(df.dtypes)
```

Bu yerda `dtype` parametrini ishlatib, har bir ustun uchun kerakli ma'lumot turini belgiladik, bu esa xotira tejashga yordam beradi.

Bo'sh qiymatlarni kamaytirish

Bo'sh (NaN) qiymatlarni to'ldirish yoki olib tashlash ham samaradorlikni oshirishga yordam beradi. Bo'sh qiymatlar bilan ishlashda `fillna()` yoki `dropna()` metodlaridan foydalanish mumkin.

Misol:

```
# NaN qiymatlarni o'zgartirish
df['column1'] = df['column1'].fillna(0)
```

```
# NaN qiymatlarni olib tashlash
df = df.dropna()
```

Bu usullar bo'sh qiymatlarni kamaytirish orqali ma'lumotlar ustida samarali ishlash imkonini beradi.

Categorical Tipidan Foydalanish

Pandasda **categorical** turi juda samarali, chunki u bir nechta takrorlanuvchi qiymatlar bilan ishlashda xotirani tejaydi. **Categorical** turidagi ustunlar, masalan, kategoriyalar, turli davlatlar nomlari yoki mahsulot turlari kabi ma'lumotlar bilan ishlashda ayniqsa foydalidir.

Categorical tipini qo'llash

1. categorical turini qo'llash:

Misol uchun, **Shahar** ustuni faqat bir nechta takrorlanuvchi shahar nomlarini o'z ichiga oladi. Bu ustunni **categorical** turiga o'zgartirish orqali xotira sarfini kamaytirish mumkin:

```
df['Shahar'] = df['Shahar'].astype('category')
print(df['Shahar'].dtype)
```

Natija:

category

Bu usul orqali **Shahar** ustuni endi kategoriyalar sifatida saqlanadi, bu esa xotirani sezilarli darajada tejashga yordam beradi.

2. Categorical turlarni guruhlash va agregatsiya qilish:

Kategoriyalangan ustunlar bilan ishlash, guruhlash va agregatsiya qilishda samarali:

```
df = pd.DataFrame({
    'Ism': ['Ali', 'Veli', 'Sami', 'Zafar', 'Sara'],
    'Shahar': ['Tashkent', 'Samarkand', 'Tashkent', 'Samarkand', 'Tashkent'],
    'Yosh': [23, 45, 34, 34, 28]
})
```

```
# 'Shahar' ustunini categoric turiga o'zgartirish
df['Shahar'] = df['Shahar'].astype('category')
```

```
# Kategoriyalarga bo'lish va o'rtacha yoshni hisoblash
df_grouped = df.groupby('Shahar')['Yosh'].mean()
print(df_grouped)
```

Natija:

```
Shahar
Samarkand    39.5
Tashkent     28.333333
Name: Yosh, dtype: float64
```

Bu yerda **Shahar** ustuni kategoriyalangan bo'lib, ma'lumotlarni guruhlash va agregatsiya qilishda samarali ishlatilmoqda.

3. **Categorical tipining afzalliklari:**

- **Xotira tejash:** Categorical turlari takrorlanuvchi qiymatlarni saqlashda xotira sarfini kamaytiradi.
- **Tezlik:** Kategoriyalangan ustunlar bilan agregatsiya qilish va guruhlash operatsiyalari tezroq amalga oshadi.

Categorical tipining ko'plab qiymatlari bilan ishlash

Agar ma'lumotlar juda ko'p takrorlanuvchi qiymatlarga ega bo'lsa, masalan, davlatlar yoki mahsulot nomlari, `categorical` turi ularni optimallashtiradi:

```
df['Country'] = pd.Categorical(df['Country'], categories=['USA', 'Canada', 'Mexico'])
```

Bu yerda **Country** ustuni faqat **USA**, **Canada**, va **Mexico** kabi kategoriyalarni o'z ichiga oladi, va Pandas bu ustunni optimal saqlaydi.

Xulosa

- **Katta hajmdagi ma'lumotlar bilan ishlash:** Pandas yordamida katta hajmdagi ma'lumotlar bilan samarali ishlash uchun `chunksize` yordamida fayllarni bo'lib o'qish, `dtype` ni optimallashtirish va bo'sh qiymatlarni kamaytirish kabi usullar mavjud.
- **Categorical tipidan foydalanish:** `categorical` turi ma'lumotlarni samarali saqlash va ishlashda juda foydalidir. Bu usul, ayniqsa, ko'p takrorlanuvchi qiymatlar bo'lgan ustunlarda xotira sarfini kamaytirish va tezlikni oshirishga yordam beradi.

9. Mashqlar va Amaliyot

Pandas yordamida ma'lumotlarni tahlil qilish bo'yicha mashqlar va kichik proyektlar o'rganishning samarali usulidir. Quyida **real hayotdan misollar** yordamida **Pandas** kutubxonasini qanday qo'llashni ko'rib chiqamiz.

Mashq 1: Savdo Ma'lumotlarini Tahlil Qilish

Vazifa: Aylanish savdolarini tahlil qiling. Ma'lumotlar jadvali quyidagi formatda bo'lishi kerak:

Sana	Mahsulot	Sotilgan son	Narx
2023-01-01	A	10	100
2023-01-01	B	15	200
2023-01-02	A	12	100
2023-01-02	B	18	200
...	...	...	...

Qadamlar:

1. **Ma'lumotlarni o'qish:**

```
import pandas as pd

# Ma'lumotlarni yaratish
data = {
    'Sana': ['2023-01-01', '2023-01-01', '2023-01-02', '2023-01-02'],
    'Mahsulot': ['A', 'B', 'A', 'B'],
    'Sotilgan son': [10, 15, 12, 18],
    'Narx': [100, 200, 100, 200]
}

df = pd.DataFrame(data)

# Sana ustunini datetime turiga o'zgartirish
df['Sana'] = pd.to_datetime(df['Sana'])

print(df)
```

2. Savdo umumiy qiymatini hisoblash:

Savdo umumiy qiymatini hisoblash uchun **Sotilgan son** va **Narx** ustunlarini ko'paytiramiz:

```
df['Savdo qiymati'] = df['Sotilgan son'] * df['Narx']
print(df)
```

Natija:

	Sana	Mahsulot	Sotilgan son	Narx	Savdo qiymati
0	2023-01-01	A	10	100	1000
1	2023-01-01	B	15	200	3000
2	2023-01-02	A	12	100	1200
3	2023-01-02	B	18	200	3600

3. Kunlik savdo umumiy qiymatini hisoblash:

Har bir kun uchun jami savdo qiymatini hisoblash uchun **groupby()** metodidan foydalanamiz:

```
daily_sales = df.groupby('Sana')['Savdo qiymati'].sum()
print(daily_sales)
```

Natija:

```
Sana
2023-01-01    4000
2023-01-02    4800
Name: Savdo qiymati, dtype: int64
```

4. Savdo bo'yicha mahsulot tahlili:

Har bir mahsulot bo'yicha jami sotilgan sonni va umumiy savdo qiymatini hisoblash:

```
product_sales = df.groupby('Mahsulot').agg({'Sotilgan son': 'sum', 'Savdo qiymati': 'sum'})
print(product_sales)
```

Natija:

	Sotilgan son	Savdo qiymati
A	22	2200
B	33	6600

Bu mashq yordamida siz **groupby()** metodini, agregatsiya funksiyalarini va **datetime** bilan ishlashni o'rgandingiz.

Mashq 2: Talablar bo'yicha Ma'lumotlarni Filtrlash

Vazifa: Mahsulotlar va ularning sotilishi bilan bog'liq ma'lumotlarni tahlil qiling. Har bir mahsulotning **narxi** va **sotilgan soni** berilgan. **Mahsulot** va **narx** ustunlari bo'yicha filtrlar qo'yib, talablarni ko'rsating.

Qadamlar:

1. Ma'lumotlarni yaratish:

```
df = pd.DataFrame({
    'Mahsulot': ['A', 'B', 'A', 'C', 'B', 'A'],
    'Narx': [100, 200, 150, 120, 250, 180],
    'Sotilgan son': [10, 5, 8, 12, 4, 9]
})
```

```
print(df)
```

Natija:

	Mahsulot	Narx	Sotilgan son
0	A	100	10
1	B	200	5
2	A	150	8
3	C	120	12
4	B	250	4
5	A	180	9

2. Narxi 150 dan yuqori bo'lgan mahsulotlarni tanlash:

```
filtered_data = df[df['Narx'] > 150]
print(filtered_data)
```

Natija:

	Mahsulot	Narx	Sotilgan son
1	B	200	5
4	B	250	4
5	A	180	9

3. Sotilgan soni 8 dan yuqori bo'lgan mahsulotlarni tanlash:

```
filtered_data = df[df['Sotilgan son'] >= 8]
print(filtered_data)
```

Natija:

	Mahsulot	Narx	Sotilgan son
0	A	100	10
2	A	150	8
3	C	120	12
5	A	180	9

Bu mashq orqali siz **DataFrame** ustunlari bo'yicha **shartlar** asosida filtrlarni qanday qo'llashni o'rgandingiz.

Mashq 3: Film Reytinglarini Tahlil Qilish

Vazifa: Film reytinglari bilan bog'liq ma'lumotlar to'plamidan foydalaning. Har bir filmning reytingi va yili berilgan. O'rta reytinglarni hisoblash va eng yuqori reytingga ega bo'lgan filmlar haqida ma'lumot oling.

Qadamlar:

1. Film reytinglari ma'lumotlarini yaratish:

```
df = pd.DataFrame({
    'Film': ['Film1', 'Film2', 'Film3', 'Film4', 'Film5'],
    'Reyting': [7.5, 8.2, 6.9, 7.8, 9.0],
    'Yil': [2021, 2022, 2021, 2020, 2022]
})
```

```
print(df)
```

Natija:

	Film	Reyting	Yil
0	Film1	7.5	2021
1	Film2	8.2	2022
2	Film3	6.9	2021
3	Film4	7.8	2020
4	Film5	9.0	2022

2. O'rta reytingni hisoblash:

```
mean_rating = df['Reyting'].mean()
print("O'rta reyting:", mean_rating)
```

Natija:

O'rta reyting: 7.68

3. Eng yuqori reytingga ega bo'lgan filmlar:

```
topRated = df[df['Reyting'] == df['Reyting'].max()]
print("Eng yuqori reytingga ega bo'lgan film:")
print(topRated)
```

Natija:

	Film	Reyting	Yil
4	Film5	9.0	2022

Bu mashq orqali siz ma'lumotlarni **tahlil qilish** va **eng yaxshi qiymatlarni** olishni o'rgandingiz.

Kichik Projektlar

- Savdo Hisobotini Yaratish:** Katta savdo ma'lumotlari to'plamidan foydalanib, har bir mahsulot va shahar bo'yicha savdo statistikalari chiqarish va hisobotni yaratish.
- Mijozlar Profilini Tahlil Qilish:** Mijozlarning xarid qilish odatlarini tahlil qilish va eng ko'p sotilgan mahsulotlar haqida hisobotlar yaratish.
- Sport Musobaqalari Reytingini Tahlil Qilish:** Sport musobaqalari va ularning reytinglarini tahlil qilib, eng yaxshi jamoalar va sportchilar haqida hisobot tayyorlash.

Xulosa

- Mashqlar** yordamida Pandas kutubxonasining asosiy funksiyalarini o'rganish va amaliyot qilish mumkin.
- Kichik projektlar** orqali real hayotdagi muammolarni Pandas yordamida tahlil qilish va ma'lumotlarni samarali tarzda manipulyatsiya qilishni o'rgandingiz.

Bu mashqlar va proyektlar sizga Pandas kutubxonasida mustahkam bilimga ega bo'lishga yordam beradi.

10. Qayta Aloqa va Izlanish

Pandas Rasmiy Dokumentatsiyasi

Pandas kutubxonasi haqida chuqurroq ma'lumot olish va uning imkoniyatlarini to'liq o'rganish uchun rasmiy **Pandas dokumentatsiyasi** juda muhim manba hisoblanadi. Pandasning rasmiy dokumentatsiyasi ko'plab misollar, metodlar, funksiyalar, va ularning ishlatilishiga oid batafsil tushuntirishlarni o'z ichiga oladi.

1. Pandasning rasmiy hujjati:

- **Havola:** <https://pandas.pydata.org/pandas-docs/stable/>

Pandas hujjatlari quyidagi asosiy bo'limlarni o'z ichiga oladi:

- **Ma'lumotlarni o'qish va yozish:** CSV, Excel, SQL, JSON va boshqa formatlar bilan ishlash.
- **Pandas kutubxonasi asoslari:** Series, DataFrame va indekslar bilan ishlash.
- **Funksiyalar va metodlar:** Pandasda mavjud bo'lgan barcha metodlar va funksiyalar haqida batafsil ma'lumotlar.
- **Ilg'or funksiyalar:** Katta hajmdagi ma'lumotlar bilan ishlash, vaqt bo'yicha ma'lumotlar, agregatsiya va guruhlash.

2. Pandas API:

- **Havola:** <https://pandas.pydata.org/pandas-docs/stable/reference/index.html>

Bu bo'limda Pandas kutubxonasining barcha metodlari va atributlari haqida so'z boradi. Har bir metodning misoli va parametrlarining batafsil tushuntirishi mavjud.

O'rganish Davomida Yuzaga Kelgan Savollarni Onlayn Forumlarda Muhokama Qilish

Pandasni o'rganish jarayonida yuzaga keladigan savollar va muammolarni hal qilishda **onlayn forumlar** va **hamkorlar bilan muhokama qilish** juda foydali bo'ladi. Quyidagi resurslar sizga savollarni so'rash va muhokama qilishda yordam beradi:

1. Stack Overflow:

- Stack Overflow — bu dunyodagi eng mashhur dasturlash forumlaridan biridir. Pandas kutubxonasiga oid savollarni topish yoki yangi savol berish uchun ideal platforma.
- **Havola:** <https://stackoverflow.com/questions/tagged/pandas>

Stack Overflow'da Pandas bilan bog'liq yuzlab savollar va javoblar mavjud. Savollaringizni **pandas** tegini qo'shgan holda joylashtiring.

2. Pandas GITHUB repository:

- Pandasning rasmiy **GitHub repository**sida kutubxona bilan bog'liq xatolarni topish, patchlar yuborish yoki boshqa foydalanuvchilar bilan muhokama qilish mumkin.
- **Havola:** <https://github.com/pandas-dev/pandas>

Bu yerda siz kutubxonadagi yangi xatoliklarni ko'rish va ularni qanday tuzatish bo'yicha tavsiyalarni topishingiz mumkin.

3. Reddit:

- Pandasni o'rganishda yordam olish uchun Redditda turli subreddits mavjud, masalan, **r/learnpython** va **r/datascience**.
- **Havola:** <https://www.reddit.com/r/learnpython/>

Reddit forumlarida foydalanuvchilar Pandas va Python dasturlash bo'yicha o'z bilimlari bilan o'rtoqlashishadi. Savollarni joylashtirish va boshqalar bilan tajriba almashish uchun yaxshi joy.

4. Pandas Google Group:

- Pandasning **Google Group** forumida kutubxona bilan bog'liq savollarni berish, muhokama qilish va yangiliklar olish mumkin.
- **Havola:** <https://groups.google.com/forum/#!forum/pandas-dev>

Bu forumda Pandas kutubxonasini ishlab chiqish bilan bog'liq yangiliklar, xatolar va foydalanuvchilarni muhokama qilish uchun yaxshi manba.

5. Pandas Documentation Issue Tracker:

- Agar Pandas dokumentatsiyasida biror muammo yoki noaniqlik bo'lsa, uni Pandasning **Issue Tracker** orqali bildirishingiz mumkin.
- **Havola:** <https://github.com/pandas-dev/pandas/issues>

Bu yerda siz kutubxona bilan bog'liq muammolarni aniqlashingiz va ularni tuzatish bo'yicha takliflar kiritishingiz mumkin.

Xulosa

- **Pandas rasmiy dokumentatsiyasi:** Pandas kutubxonasini o'rganish jarayonida rasmiy dokumentatsiyani **o'rganish** juda muhim. Bu dokumentatsiya Pandasning barcha imkoniyatlarini tushunishda va samarali ishlatishda yordam beradi.
- **Onlayn forumlar va muhokama:** O'rganish davomida yuzaga kelgan savollarni **Stack Overflow**, **Reddit**, va **Pandas GitHub** kabi onlayn forumlar orqali muhokama qilish mumkin. Bu sizga tezda javob olish va boshqa foydalanuvchilar bilan fikr almashish imkonini beradi.

Bu manbalar va platformalar orqali siz o'rganish jarayonini yanada samarali va tezroq amalga oshirishingiz mumkin.

In []: